Design and Implementation of Embedded Systems Applications Using STM32: A Practical Approach to GPIO, ADC, Timers, Displays, and Motor Control

Project synopsis submitted in partial fulfillment
for the Award of
CERTIFICATION OF COMPLETION
in
Certification Course in Embedded Systems Design with STM32 & ARM Cortex-M

By

Dhananjay Sunil Matkar

DIYguru Mobility Pvt. Ltd
(Empanelled by NEAT, AICTE, Ministry of Education, Gov. of India)

DATE:02/11/2026

Abstract

This project report presents a practical introduction to embedded-system development using an STM32 microcontroller based on the ARM Cortex-M architecture. Seven related tasks are documented: GPIO LED control, a two-digit seven-segment counter, LCD custom-character generation, ADC measurement and LCD display, interrupt-driven timer operation, servo-motor positioning, and a UART communication framework. Each task connects a hardware interface to a clear firmware design and a repeatable test procedure.

The examples use the STM32CubeIDE workflow and STM32 Hardware Abstraction Layer (HAL) APIs. Exact pin assignments, peripheral clock settings, and timer parameters depend on the selected STM32 part and board; therefore, the report states its assumptions and explains how to verify them in the generated project. The code snippets are intended as educational starting points and should be integrated with the initialization code produced by STM32CubeMX/CubeIDE.

The overall emphasis is on modular C code, readable comments, debugging, electrical safety, measurement, and validation. Together, the tasks demonstrate how simple peripherals can be combined into reliable embedded applications.

Keywords

STM32, ARM Cortex-M, embedded C, GPIO, ADC, seven-segment display, LCD, timer interrupt, PWM, servo motor, UART, debugging.

How to use this report

Read the platform assumptions first, then configure the matching MCU and pins in CubeIDE. Build and test each task independently before combining modules. Record measured values and screenshots from the actual hardware in the final submission; do not report simulated or expected values as measured results.

Table of Contents

Section	Pages
1. Introduction and scope	4–5
2. Platform, tools and safety	6–7
3. Task 1 — LED using GPIO	8–11
4. Task 2 — Double seven-segment display	12–16
5. Task 3 — LCD custom characters using CGRAM	17–21
6. Task 4 — ADC with LCD output	22–26
7. Task 5 — Interrupt-driven timer	27–30
8. Task 6 — Servo motor control	31–34
9. Task 7 — UART communication framework	35–37
10. Integration, debugging and evaluation	38
11. Challenges and conclusion	39
12. References and appendices	40

Note: Page numbers are arranged as a 40-page teaching-report structure. If this report is edited, pagination may shift slightly depending on font rendering and the final printer settings.

1. Introduction

Embedded systems are dedicated computing systems designed to monitor, control, or communicate with a physical process. Unlike a general-purpose computer, an embedded controller typically operates under constraints such as limited memory, real-time deadlines, low power consumption, and direct electrical interaction with sensors and actuators.

STM32 microcontrollers are a family of 32-bit MCUs developed by STMicroelectronics. Many STM32 devices use ARM Cortex-M cores and include GPIO ports, timers, ADCs, serial interfaces, DMA controllers, and other peripherals. The exact feature set differs by family and part number, so firmware must be configured for the selected device rather than copied blindly between boards.

Why practical peripheral work matters

A peripheral becomes useful when firmware configures it correctly, reads or writes meaningful values, and handles timing and errors predictably. The LED task introduces digital output. The seven-segment and LCD tasks develop display-driving logic. The ADC task converts a voltage into a numerical measurement. Timer interrupts introduce periodic execution, PWM provides a timed control signal for a servo, and UART allows data exchange with another device or a terminal.

Project scope

The report treats the seven listed deliverables as independent modules. The examples assume a common STM32Cube HAL project structure with main.c, peripheral initialization functions, and user code sections. Students should adapt names and pins to the code generated for their own board.

1.1 Objectives and expected outcomes

Objective	Evidence of completion
Understand STM32 programming	Project builds in STM32CubeIDE and runs on the selected board.
Apply theory practically	GPIO, ADC, timer, PWM, display, and UART interfaces are demonstrated.
Write and document embedded C	Functions are modular, commented, and use meaningful names.
Debug systematically	Clock, pin, wiring, and signal checks are recorded.
Solve practical problems	Each task has test cases and corrective actions.

Deliverables

1. LED control with GPIO. 2. A two-digit display counting 00–99. 3. LCD custom characters written to CGRAM. 4. ADC reading of an analog input shown on an LCD. 5. Interrupt-driven timer event. 6. Servo rotation in both directions. 7. UART communication framework.

Success criteria

A task is considered complete when the circuit is wired safely, the firmware compiles without errors, the expected output is observed on hardware, and the student can explain the code path and key configuration settings. Screenshots, serial logs, and measured values should be added where required by the instructor.

2. Hardware and software platform

Reference platform: an STM32 development board with a supported STM32 MCU, STM32CubeIDE, STM32CubeMX device configuration, and HAL drivers. The code is written in portable C style, but peripheral instance names and pin mappings must match the selected chip.

Item	Purpose / note
STM32 development board	Target MCU and regulated logic supply.
USB cable / ST-LINK	Programming, debugging, and sometimes virtual COM access.
Breadboard and jumper wires	Temporary low-voltage circuit connections.
LED and current-limiting resistor	GPIO output demonstration; typically 220–1,000 Ω depending on supply and LED.
Two-digit seven-segment display	Common-cathode or common-anode type; confirm before wiring.
16×2 character LCD	HD44780-compatible module or I²C backpack, depending on chosen interface.
Potentiometer	ADC test input, wired between 3.3 V and GND with wiper to ADC input.
Servo motor	Position-controlled hobby servo; external supply may be needed.
USB-to-UART adapter / terminal	Serial data verification if a virtual COM port is not available.

Before wiring, check the exact board schematic, pinout, maximum GPIO current, ADC input limits, and peripheral alternate-function mapping for the selected MCU.

2.1 Development workflow and electrical safety

Firmware workflow

1. Create a project for the exact MCU in STM32CubeIDE. 2. Configure the system clock and required peripherals in the .ioc editor. 3. Generate initialization code. 4. Add application logic inside user-code sections or separate source files. 5. Build and resolve warnings. 6. Flash the board. 7. Verify behavior with the debugger, multimeter, oscilloscope, logic analyzer, or serial terminal.

Electrical precautions

Use the board's specified logic voltage; many STM32 GPIO pins operate at 3.3 V and are not 5 V tolerant. Never connect a 5 V signal to an input unless the specific pin's datasheet explicitly permits it. Share ground between the MCU and external servo supply. Do not power a servo from a GPIO pin. Use a suitable regulated supply and check stall current. Use a current-limiting resistor with a bare LED. Disconnect power before changing wiring.

Configuration note: STM32 HAL function names below are representative. A generated project may use different peripheral handles or pin definitions. Always confirm the selected MCU reference manual and datasheet.

3. Task 1 — LED control using GPIO

Objective

Configure a GPIO pin as a digital output and use firmware to switch an LED on and off at a visible interval. This task introduces GPIO initialization, output state changes, timing delays, and basic debugging.

Theory

A GPIO output drives a logic level. In push-pull mode, the pin actively drives both high and low states. The LED must be connected with correct polarity and a series resistor. Depending on the board, the built-in LED may be active-high or active-low; verify the board documentation.

Example connection

Signal	Connection
LED anode	GPIO output through a suitable series resistor, for active-high wiring.
LED cathode	GND.
Alternative	Use the board's built-in LED and its documented pin/polarity.

Configuration

In CubeIDE, set the selected pin to GPIO_Output, output type Push-Pull, no pull-up/down unless needed, and a suitable low/medium speed. Generate code and confirm the GPIO port and pin macros.

3.1 LED firmware

```
/* Example application logic for STM32 HAL.
 * Replace LED_GPIO_Port and LED_Pin with generated names.
 */
while (1)
{
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
    HAL_Delay(500);  /* LED on for approximately 500 ms */

    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
    HAL_Delay(500);  /* LED off for approximately 500 ms */
}
```

Logic explanation

The infinite loop repeats two states. GPIO_PIN_SET drives the output high and GPIO_PIN_RESET drives it low. HAL_Delay uses the HAL tick, normally generated by a SysTick interrupt. The example produces an approximate one-second full blink period, with a 50% duty cycle.

Alternative toggle approach

```
while (1)
{
    HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
    HAL_Delay(500);
}
```

The toggle version is shorter, but explicit set/reset operations can be easier to read when a specific output state is required.

3.2 Build, test and debugging

Test procedure

1. Confirm the correct target MCU and debug interface. 2. Build the project. 3. Flash and start execution. 4. Observe the LED for at least ten cycles. 5. Change the delay to 100 ms and 1,000 ms to verify that the blink rate responds as expected. 6. Restore the desired value and record the result.

Symptom	Likely cause	Check / remedy
LED never turns on	Wrong pin, wiring, polarity, or GPIO clock not	enCahbelecdk.board schematic, generated macros, and GPIO initia lization.
LED stays on	Code not reaching reset branch or active-low	LEUDs.e debugger breakpoints; verify polarity.
Blink rate differs	Clock/tick configuration or delay not as expect	dC.heck HAL time base and system clock configuration.
Build error on pin macro	Macro name differs from example.	Use the names generated by CubeMX.

Expected result

The LED alternates between on and off states at the selected interval. Record the actual board, pin, delay, and whether the LED is active-high or active-low.

3.3 GPIO concepts and extension

GPIO configuration includes mode, output type, speed, and pull resistors. Input pins can be used for buttons or sensors; output pins can drive LEDs or logic inputs, within the limits specified in the datasheet. A mechanical push button often needs a pull-up or pull-down resistor and debouncing.

Suggested extension: button-controlled LED

```
/* Example only: BTN_Pin is configured as an input.
 * A pull-up button typically reads RESET when pressed
 * if wired from the pin to GND.
 */
if (HAL_GPIO_ReadPin(BTN_GPIO_Port, BTN_Pin) == GPIO_PIN_RESET)
{
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
}
else
{
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
}
```

This simple example does not debounce the button. For a robust implementation, sample the input periodically and require a stable state for a defined interval, or use an interrupt followed by software filtering.

Task 1 evidence checklist: wiring photograph, CubeIDE pin configuration, successful build, observed blink pattern, and a short note describing any issue and its resolution.

4. Task 2 — Double seven-segment display

Objective

Display decimal numbers from 00 through 99 using two seven-segment digits. The firmware converts a counter value into tens and units digits and drives the display segments. A multiplexed design is commonly used to reduce GPIO usage.

Display principle

Each digit contains segments a–g and often a decimal point. A lookup table maps each decimal digit to a segment pattern. In a multiplexed display, the firmware activates one digit at a time rapidly; persistence of vision makes both digits appear continuously illuminated.

Important hardware distinction

Common-cathode displays turn a segment on when its segment pin is driven high (subject to the driver circuit). Common-anode displays generally use the opposite segment polarity. Digit-enable polarity also depends on the circuit. Use transistor drivers where required and avoid exceeding MCU pin current limits.

Approach	Advantages	Trade-offs
Direct drive	Simple logic.	Uses many GPIO pins; current must be limited.
Multiplexed	Fewer pins for two digits.	Requires fast refresh and correct digit switching.
Driver IC	Simplifies current and scan control.	Adds hardware and interface setup.

4.1 Segment map and digit table

The following table uses bit positions a–g in bits 0–6 and assumes a common-cathode, active-high segment interface. Wiring order must match the bit mapping. For common-anode hardware, invert the segment pattern or implement an explicit polarity layer.

Digit	g f e d c b a (conceptual)	Hex pattern (bit 0 = a)
0	abcdef	0x3F
1	bc	0x06
2	abdeg	0x5B
3	abcdg	0x4F
4	bcfg	0x66
5	acdfg	0x6D
6	acdefg	0x7D
7	abc	0x07
8	abcdefg	0x7F
9	abcdfg	0x6F

Design decisions

For a classroom circuit, assign seven segment lines to GPIO outputs and two digit-enable lines to separate GPIOs. Add one current-limiting resistor per segment line, or use an appropriate driver circuit. If both digits are enabled simultaneously, current may exceed safe limits and ghosting may occur.

4.2 Multiplexing algorithm

A reliable multiplex routine performs these steps: blank both digits, write the segment pattern for the desired digit, enable only that digit, wait briefly, then repeat for the other digit. The refresh function should be called often enough to avoid visible flicker. A hardware timer interrupt is preferable to long blocking delays in a larger application.

```
static const uint8_t segCode[10] = {
    0x3F, 0x06, 0x5B, 0x4F, 0x66,
    0x6D, 0x7D, 0x07, 0x7F, 0x6F
};

void Display_Number(uint8_t value)
{
    uint8_t tens = value / 10U;
    uint8_t ones = value % 10U;

    /* These helpers depend on the actual wiring/polarity. */
    Digit_AllOff();
    Segments_Write(segCode[tens]);
    Digit_TensOn();
    HAL_Delay(2);

    Digit_AllOff();
    Segments_Write(segCode[ones]);
    Digit_OnesOn();
    HAL_Delay(2);

    Digit_AllOff();
}
```

This simple routine illustrates the algorithm, but a blocking delay-based implementation can disrupt other time-critical tasks. For continuous display output, use a periodic timer interrupt or a non-blocking state machine.

4.3 Counting from 00 to 99

```
uint8_t count = 0U;
uint32_t lastStep = 0U;

while (1)
{
    /* Refresh the display continuously. */
    Display_Number(count);

    /* Increment once per second without a long blocking delay. */
    if ((HAL_GetTick() - lastStep) &gt;= 1000U)
    {
        lastStep = HAL_GetTick();
        count++;
        if (count &gt; 99U)
        {
            count = 0U;
        }
    }
}
```

For a production-quality design, separate display refresh from the counter timebase. The display should be scanned continuously, while the count changes only once per second. The sample is conceptual because Display_Number contains delays; replace it with a non-blocking scan routine before using it alongside communication or control tasks.

Recommended state-machine structure

Maintain a counter value, the currently selected digit, and a refresh timestamp. Each refresh event updates one digit and toggles the digit index. A separate one-second event increments the counter. This avoids long delays and improves responsiveness.

4.4 Validation and common faults

Test	Expected behavior
Power-on	Both digits show 00 or a documented startup value.
Increment	Display changes once per second: 00, 01, 02 … 99.
Wraparound	After 99, the display returns to 00.
Digit scan	Both digits appear steady with no obvious flicker.
Polarity test	Only intended segments illuminate.

Troubleshooting

If digits are reversed, swap digit-enable assignments. If segment patterns are incorrect, confirm the physical pin order and bit mapping. If the display is dim, check resistor values, duty cycle, supply voltage, and driver current. Ghosting can result from enabling the next digit before disabling the previous one. Flicker indicates an insufficient refresh rate or excessive time spent in other code.

Record whether the display is common-anode or common-cathode, the segment pin map, the scan interval, and the observed 00–99 wraparound behavior.

5. Task 3 — LCD custom characters using CGRAM

Objective

Create custom characters and display them on a character LCD. Most HD44780-compatible 5×8 character LCD modules provide Character Generator RAM (CGRAM) locations for user-defined glyphs. Each custom glyph is stored as a set of five-bit row patterns.

LCD interface options

A parallel LCD interface commonly uses RS, E, and data lines D4–D7 in 4-bit mode. An I²C backpack reduces the number of MCU pins but requires a compatible driver and correct address. This report demonstrates the custom-character data and high-level API; the exact low-level write functions depend on the LCD library used.

Signal	Typical function
RS	0 for command; 1 for character data.
E	Enable strobe to latch a command or data nibble/byte.
D4–D7	Four-bit data bus in 4-bit mode.
R/W	Often tied to GND for write-only operation.
V0 / contrast	Contrast adjustment; follow module datasheet.

5.1 CGRAM data format

For a 5×8 glyph, each of the eight bytes defines one row. Only the lower five bits are used; bit 4 is the leftmost pixel in the usual representation. A row value such as 0b00100 lights the center pixel. Custom character codes are typically 0–7, though exact behavior depends on the LCD controller.

```
/* Example 5x8 heart glyph. */
uint8_t heart[8] = {
    0b00000,
    0b01010,
    0b11111,
    0b11111,
    0b11111,
    0b01110,
    0b00100,
    0b00000
};
```

Library interface assumption

The example below assumes helper functions LCD_CreateChar(slot, bitmap), LCD_SetCursor(row, column), and LCD_Print(text). These are illustrative application-level functions, not universal STM32 HAL functions. If using a specific LCD library, map these calls to its API.

CGRAM addressing concept

To define character slot n, the driver sends a Set CGRAM Address command followed by eight row bytes. It then sends a DDRAM address command to return to the display text area before printing ordinary text.

5.2 LCD custom-character code

```
/* Application-level example. Replace the LCD helper functions
 * with the API supplied by your chosen LCD driver.
 */
uint8_t heart[8] = {
    0x00, 0x0A, 0x1F, 0x1F,
    0x1F, 0x0E, 0x04, 0x00
};

uint8_t smile[8] = {
    0x00, 0x0A, 0x00, 0x00,
    0x11, 0x0E, 0x00, 0x00
};

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();

    LCD_Init();
    LCD_CreateChar(0, heart);
    LCD_CreateChar(1, smile);

    LCD_SetCursor(0, 0);
    LCD_Print("STM32 LCD Demo");
    LCD_SetCursor(1, 0);
    LCD_WriteChar(0);  /* custom character in slot 0 */
    LCD_Print("  Welcome ");
    LCD_WriteChar(1);  /* custom character in slot 1 */

    while (1) {
        /* Application can update other LCD fields here. */
    }
}
```

The main function shows the intended call sequence: initialize the MCU, initialize the LCD, load glyphs into CGRAM, position the cursor, and write text and custom-character codes.

5.3 LCD driver logic and timing

For a 4-bit interface, each byte is transferred as two nibbles. The driver places the upper nibble on D4–D7, sets RS, pulses E, then transfers the lower nibble and pulses E again. Some commands require longer execution time than ordinary data writes, especially clear-display and return-home commands.

```
/* Pseudocode for writing a byte in 4-bit mode.
 * Implement pin writes and timing for the actual circuit.
 */
void LCD_SendByte(uint8_t value, bool isData)
{
    LCD_RS_Write(isData);
    LCD_WriteNibble((value &gt;&gt; 4) &amp; 0x0F);
    LCD_EnablePulse();
    LCD_WriteNibble(value &amp; 0x0F);
    LCD_EnablePulse();
    LCD_WaitCommandTime();
}
```

Implementation notes

Initialize the LCD according to its controller datasheet. Check whether the library expects row/column order as (row, column) or (column, row). Do not assume custom glyphs persist after power loss; CGRAM contents are volatile and should be loaded at startup.

5.4 Test plan and faults

Test	Expected result
Initialization	Text is visible and contrast is readable.
Glyph slot 0	Heart glyph appears where code 0 is written.
Glyph slot 1	Smile glyph appears where code 1 is written.
Power cycle	Firmware reloads glyphs at startup.

Troubleshooting

A blank screen may be caused by contrast adjustment, power wiring, or incorrect initialization. Blocks on the first row often indicate power is present but the controller has not initialized. Garbled characters may indicate incorrect nibble order, wrong RS/E wiring, or timing that is too short. If custom characters appear as unrelated symbols, check the slot index and CGRAM command sequence.

Use an oscilloscope or logic analyzer on RS, E, and D4–D7 if available. Verify that E pulses occur and the command/data mode is correct. If using I²C, scan or verify the documented address and check SDA/SCL pull-ups.

6. Task 4 — ADC measurement with LCD output

Objective

Read an analog voltage, such as the wiper output of a potentiometer, using the STM32 ADC and display the converted reading on an LCD. The task demonstrates analog input limits, ADC resolution, conversion, scaling, and user-readable output.

ADC concept

An analog-to-digital converter maps an input voltage within its configured range to an integer code. For a 12-bit ADC, the nominal code range is 0–4095. The estimated voltage is V_in ≈ ADC_code × V_ref / 4095, assuming a 12-bit conversion and the correct reference voltage.

> Never exceed the selected ADC pin's allowed voltage. On many STM32 boards the ADC input range is 0 V to VDDA (often about 3.3 V), not 5 V. Confirm the datasheet and board design.

Potentiometer wiring

Potentiometer terminal	Connection
Outer terminal 1	3.3 V / VDDA as allowed by the board.
Outer terminal 2	GND.
Center wiper	Selected ADC-capable input pin.

6.1 ADC configuration

In CubeMX, enable an ADC instance and select a channel mapped to the chosen physical pin. Configure resolution, data alignment, sampling time, and continuous or single conversion mode according to the application. The example below uses a blocking single conversion for clarity.

Assumptions

The generated project provides hadc1 and MX_ADC1_Init(). The selected ADC channel and GPIO analog mode are configured in the .ioc file. LCD functions are application-level helpers from the chosen LCD driver.

```
uint32_t ADC_ReadOnce(void)
{
    uint32_t value = 0U;

    if (HAL_ADC_Start(&amp;hadc1) == HAL_OK)
    {
        if (HAL_ADC_PollForConversion(&amp;hadc1, 20U) == HAL_OK)
        {
            value = HAL_ADC_GetValue(&amp;hadc1);
        }
        (void)HAL_ADC_Stop(&amp;hadc1);
    }
    return value;
}
```

The function starts a conversion, waits for completion with a timeout, reads the conversion register, and stops the ADC. A robust application should distinguish a valid reading from a timeout or peripheral error rather than silently treating every failure as zero.

6.2 Convert ADC code to voltage

```
#define ADC_MAX_CODE  4095.0f
#define ADC_VREF      3.300f  /* Replace with measured VDDA/VREF+ */

float ADC_CodeToVoltage(uint32_t raw)
{
    return ((float)raw * ADC_VREF) / ADC_MAX_CODE;
}
```

The calculation assumes a 12-bit ADC and a reference of 3.300 V. For accurate measurement, use the actual reference voltage and account for the MCU's ADC characteristics, input source impedance, sampling time, noise, and calibration requirements.

Optional percentage scaling

```
uint32_t ADC_CodeToPercent(uint32_t raw)
{
    if (raw &gt; 4095U) raw = 4095U;
    return (raw * 100U) / 4095U;
}
```

Percentage scaling is useful for a potentiometer interface. It does not represent a calibrated physical quantity unless the input circuit and reference are known.

6.3 ADC and LCD application loop

```
char line[17];

while (1)
{
    uint32_t raw = ADC_ReadOnce();
    float voltage = ADC_CodeToVoltage(raw);
    uint32_t percent = ADC_CodeToPercent(raw);

    LCD_SetCursor(0, 0);
    snprintf(line, sizeof(line), "ADC: %4lu     ",
             (unsigned long)raw);
    LCD_Print(line);

    LCD_SetCursor(1, 0);
    snprintf(line, sizeof(line), "%1.2f V %3lu%%   ",
             (double)voltage, (unsigned long)percent);
    LCD_Print(line);

    HAL_Delay(200);
}
```

The LCD is updated five times per second. Padding spaces overwrite old characters when a shorter string replaces a longer one. Some embedded C libraries use reduced printf support; floating-point formatting may need to be enabled in linker settings. If not available, display integer millivolts instead.

Alternative integer-millivolt calculation

```
uint32_t millivolts = (raw * 3300U) / 4095U;
/* Display millivolts as whole-number text to avoid float printf. */
```

6.4 ADC validation and error analysis

Input condition	Expected result
Wiper near GND	ADC reading near 0; voltage near 0 V.
Wiper near midpoint	ADC reading around half scale; voltage near half reference.
Wiper near 3.3 V	ADC reading near full scale; voltage near reference.
Slow rotation	Display changes smoothly, with small noise possible.

Sources of measurement error

ADC readings can vary due to reference noise, power-supply ripple, high source impedance, poor grounding, electrical interference, or insufficient sampling time. Improve stability by routing analog wiring away from switching signals, using suitable decoupling, choosing an adequate sample time, and averaging multiple readings if response speed permits.

```
uint32_t ADC_ReadAverage(uint8_t samples)
{
    uint32_t sum = 0U;
    if (samples == 0U) return 0U;

    for (uint8_t i = 0U; i &lt; samples; i++)
    {
        sum += ADC_ReadOnce();
    }
    return sum / samples;
}
```

Averaging reduces some random noise but does not correct a wrong reference voltage or systematic calibration error.

7. Task 5 — Interrupt-driven timer

Objective

Use an STM32 hardware timer to generate periodic events through an interrupt. This task introduces timer prescalers, auto-reload values, callback functions, and the difference between interrupt work and main-loop work.

Timer operation

A timer counter increments from its input clock divided by a prescaler. When the counter reaches its auto-reload value, an update event can set a status flag and generate an interrupt. The HAL callback HAL_TIM_PeriodElapsedCallback() is called for a configured timer interrupt.

Timing calculation

For a basic up-counter, the approximate update frequency is f_update = f_timer / ((PSC + 1) × (ARR + 1)). The timer clock may not equal the CPU clock because APB prescalers and timer-clock multipliers can affect it. Verify the clock tree and reference manual.

Do not copy prescaler and auto-reload values without checking the actual timer input clock. An incorrect assumption produces a wrong interval even if the code compiles.

7.1 Timer configuration in CubeIDE

Select a general-purpose timer, enable the internal clock, choose an appropriate prescaler and period, and enable the timer global interrupt in NVIC. Generate code. Start the timer in interrupt mode using HAL_TIM_Base_Start_IT(&htimX;). The exact timer instance depends on the MCU.

Setting	Purpose
Prescaler (PSC)	Divides the timer input clock.
Counter period / ARR	Sets the number of counter ticks before update.
NVIC interrupt	Allows the timer update event to invoke the ISR path.
HAL timer handle	Identifies the configured timer instance in HAL calls.

Example design

Assume the timer clock has been confirmed as 72 MHz. For a nominal 1 kHz counter tick, PSC = 71 gives 1 MHz timer counter frequency. ARR = 999 gives an update event every 1 ms. This example is illustrative; verify the actual timer clock for your MCU and clock tree.

7.2 Interrupt-driven firmware

```
volatile uint32_t g_millisecondTicks = 0U;
volatile uint8_t g_secondEvent = 0U;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim-&gt;Instance == TIM2)
    {
        g_millisecondTicks++;

        if ((g_millisecondTicks % 1000U) == 0U)
        {
            g_secondEvent = 1U;
        }
    }
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_TIM2_Init();

    HAL_TIM_Base_Start_IT(&amp;htim2);

    while (1)
    {
        if (g_secondEvent != 0U)
        {
            g_secondEvent = 0U;
            HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        }
        /* Other non-blocking application work can run here. */
    }
}
```

The callback should remain short. It updates a counter and sets a flag; the main loop performs the LED action. The volatile qualifier prevents the compiler from assuming that the variables cannot change asynchronously. For more complex shared data, consider atomicity and critical sections.

7.3 Timer debugging and timing verification

Verification procedure

1. Confirm timer clock frequency from the clock tree. 2. Verify PSC and ARR values in the generated initialization. 3. Set a breakpoint in HAL_TIM_PeriodElapsedCallback. 4. Check that the callback is reached repeatedly. 5. Toggle a GPIO and measure its period using an oscilloscope or logic analyzer. 6. Compare the measured period with the calculated value.

Problem	Potential cause
Callback never runs	Interrupt not enabled, timer not started in IT mode, wrong handle, or IRQ handler missin g.
Interval is incorrect	Timer clock or prescaler calculation is wrong.
Application freezes	Too much work inside interrupt, blocking calls, or interrupt priority issue.
Missed events	Flag-based events can collapse repeated events if the main loop is delayed.

For high-rate events, use a counter or queue rather than a single flag. Avoid HAL_Delay inside an interrupt callback because the HAL tick may depend on an interrupt that cannot execute while the current interrupt is active.

8. Task 6 — Servo motor control

Objective

Generate a PWM control signal to command a hobby servo to move between positions in both directions. Servo position is typically controlled by pulse width repeated at a low frequency, often near 50 Hz, but exact pulse limits vary by model.

Servo interface

A common servo has power, ground, and signal leads. The signal lead receives a periodic pulse, often around 1–2 ms within a roughly 20 ms period. These values are common starting points, not guaranteed safe limits. Consult the servo datasheet and avoid forcing the horn against mechanical stops.

Servo lead	Connection
Signal	STM32 timer PWM output pin configured for the selected timer channel.
Ground	Common ground with STM32.
Power	Suitable regulated supply matching servo rating; not a GPIO output.

Power the servo from a supply capable of its startup and stall current. Share ground with the MCU, and do not assume the development board's 3.3 V regulator can power the motor.

8.1 PWM timing calculation

PWM frequency is determined by the timer input clock, prescaler, and auto-reload register. For example, with a confirmed 72 MHz timer clock, PSC = 71 produces a 1 MHz counter clock. ARR = 19999 then gives a 20 ms period (50 Hz). A compare value of 1000 corresponds to a 1 ms high pulse when the timer count and PWM mode are configured as assumed.

Parameter	Illustrative value	Meaning
Timer clock	72 MHz	Must be confirmed for the selected timer.
PSC	71	Counter clock becomes 1 MHz.
ARR	19999	Period is 20,000 counter ticks = 20 ms.
CCR	1000–2000	Approx. 1–2 ms pulse-width range.

Important note

Some servos support a wider pulse range; others may be damaged by it. Start conservatively and test the actual servo. The timer PWM pin must be mapped to the correct alternate function in CubeMX.

8.2 Servo control code

```
/* Assumes TIM3 CH1 is configured for PWM with a 20 ms period.
 * Replace htim3 and channel with the generated configuration.
 */
static void Servo_SetPulseUs(uint16_t pulse_us)
{
    if (pulse_us &amp;lt; 1000U) pulse_us = 1000U;
    if (pulse_us &amp;gt; 2000U) pulse_us = 2000U;

    __HAL_TIM_SET_COMPARE(&amp;amp;htim3, TIM_CHANNEL_1, pulse_us);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_TIM3_Init();

    HAL_TIM_PWM_Start(&amp;amp;htim3, TIM_CHANNEL_1);

    while (1)
    {
        /* Approximate one end of travel. */
        Servo_SetPulseUs(1000U);
        HAL_Delay(1000);

        /* Approximate the other end of travel. */
        Servo_SetPulseUs(2000U);
        HAL_Delay(1000);

        /* Approximate center position. */
        Servo_SetPulseUs(1500U);
        HAL_Delay(1000);
    }
}
```

The function assumes one timer tick equals one microsecond. If the timer is configured differently, convert pulse width to timer counts rather than writing microseconds directly. Use the generated timer configuration to verify the assumption.

8.3 Clockwise and counterclockwise motion

A standard positional servo does not normally provide unlimited continuous rotation. A pulse width commands a target angle; moving from a larger pulse to a smaller pulse moves the horn in the opposite direction. The relationship between pulse width and angle is servo-specific and may not be perfectly linear.

Test sequence

1. Remove mechanical load from the servo. 2. Start at the center pulse, commonly near 1.5 ms. 3. Move in small steps toward a lower pulse width. 4. Return to center. 5. Move in small steps toward a higher pulse width. 6. Confirm that the mechanism does not bind or hit a hard stop. 7. Record the usable pulse range.

Troubleshooting

Jitter may result from an undersized supply, poor grounding, electrical noise, or an incorrect PWM frequency. No motion may indicate the wrong alternate-function pin, PWM not started, incorrect timer period, or insufficient power. Resetting the MCU when the servo starts usually points to supply droop or grounding problems.

For a continuous-rotation servo, pulse width generally controls speed and direction rather than absolute angle. State which servo type was used in the final report.

9. Task 7 — UART communication framework

Objective

Implement a UART-based communication framework using an ARM Cortex-M STM32 controller. UART provides asynchronous serial communication using configured baud rate, data bits, parity, and stop bits. It is useful for debug logs, sensor readings, command input, and device-to-device links.

Typical configuration

Parameter	Common teaching setting
Baud rate	115200 bit/s (both endpoints must match).
Data bits	8.
Parity	None.
Stop bits	1.
Flow control	None unless required by the interface.
Physical connection	TX to other device RX; RX to other device TX; common ground.

UART logic levels are not the same as RS-232 voltage levels. Connect STM32 UART pins to a compatible TTL-level USB-UART adapter or the board's virtual COM interface, not directly to an RS-232 port without a level converter.

9.1 UART transmit and receive

```
/* Assumes huart2 is generated by CubeMX.
 * For simple demonstrations, blocking calls are acceptable.
 */
static const uint8_t greeting[] = "STM32 UART ready\r\n";
uint8_t rxByte;

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();

    HAL_UART_Transmit(&amp;huart2, (uint8_t *)greeting,
                      sizeof(greeting) - 1U, 100U);

    while (1)
    {
        if (HAL_UART_Receive(&amp;huart2, &amp;rxByte, 1U, 100U) == HAL_OK)
        {
            /* Echo received byte back to the terminal. */
            (void)HAL_UART_Transmit(&amp;huart2, &amp;rxByte, 1U, 100U);
        }
    }
}
```

Open a serial terminal with matching parameters. Type characters and confirm that the MCU echoes them. This blocking one-byte receive is easy to understand, but it can stall the main loop until a character arrives or the timeout expires.

More scalable design

For responsive applications, use interrupt-driven or DMA-based UART reception with a ring buffer. Parse complete commands in the main loop, validate lengths and formats, and avoid doing lengthy processing inside the UART interrupt callback.

9.2 UART message framing and robustness

UART transports a byte stream; it does not inherently define message boundaries. A framework should define how the receiver identifies a complete message. A simple text protocol can use newline termination. Binary protocols often use a start marker, length field, payload, and checksum or CRC.

```
/* Simple line-oriented parsing concept:
 * Receive bytes into rxBuffer until '\n', then process.
 * Always check buffer bounds and handle overflow.
 */
if (rxByte == '\n')
{
    rxBuffer[rxIndex] = '\0';
    ProcessCommand(rxBuffer);
    rxIndex = 0U;
}
else if (rxIndex &lt; (sizeof(rxBuffer) - 1U))
{
    rxBuffer[rxIndex++] = (char)rxByte;
}
else
{
    /* Overflow: discard partial message and reset parser. */
    rxIndex = 0U;
}
```

Suggested command examples

LED ON — turn an LED on. LED OFF — turn it off. ADC? — return the latest ADC reading. STATUS — return a short device status. Commands should be case-normalized or documented, and invalid commands should return a clear error response.

Validation

Test the correct baud rate, disconnected cable, malformed commands, long lines, and repeated messages. Confirm that the parser never writes beyond its receive buffer and that serial errors do not lock the application.

10. Integration, debugging and evaluation

After each task has been verified independently, the modules can be integrated into a single application. Integration requires careful time management: display refresh, ADC sampling, timer events, servo PWM, and UART communication must not block one another unnecessarily.

Recommended architecture

Layer	Responsibility
Board configuration	Clock, GPIO, ADC, timers, UART, and LCD driver initialization.
Drivers	Functions for LED, display, LCD, ADC, servo, and UART.
Application logic	State machine, counter, UI updates, and command handling.
Diagnostics	Error counters, status messages, and debug output.

Debugging checklist

• Verify the selected MCU, clock tree, pin mapping, and alternate functions. • Confirm the supply and common ground. • Build with warnings enabled. • Use breakpoints and inspect peripheral registers. • Measure physical signals where possible. • Change one variable at a time. • Record the observed symptom, hypothesis, test, and outcome.

Evaluation evidence

Include a photograph or screenshot for each working task, relevant CubeIDE configuration views, build status, serial output where applicable, and measured timing or voltage results. Mark any values that are calculated rather than measured.

11. Challenges faced and conclusion

Common challenges and resolutions

Challenge	Resolution approach
Incorrect GPIO mapping	Compare the generated pin macros with the board schematic.
Display shows wrong segments	Verify common-anode/cathode type, bit map, and digit-enable polarity.
LCD custom glyphs fail	Check CGRAM slot, initialization sequence, nibble order, and timing.
ADC values fluctuate	Check input range, grounding, reference voltage, sampling time, and noise.
Timer period is wrong	Recalculate using the actual timer clock and PSC/ARR values.
Servo jitters or resets board	Use an adequate external supply and common ground; verify PWM.
UART output is unreadable	Match baud rate and serial format; check TX/RX cross-connection.

Conclusion

The project demonstrates a practical path from simple digital output to analog measurement, time-based control, actuation, display interfacing, and serial communication on STM32. Each task reinforces the importance of matching firmware configuration to hardware, structuring code into reusable functions, and validating assumptions with measurement.

The most important outcome is not only that a peripheral works, but that the implementation can be explained, tested, and debugged. Students should complete the report with their own hardware details, observed outputs, photographs, and measured values to provide evidence of the implementation.

12. References and appendices

References

1. STMicroelectronics, STM32 microcontroller datasheet for the exact device used. Consult the official STMicroelectronics product page for the selected part.

2. STMicroelectronics, Reference Manual for the selected STM32 family; use it for GPIO, ADC, timer, alternate-function, and USART/UART register details.

3. STMicroelectronics, STM32CubeIDE user guide and STM32CubeMX documentation.

4. STMicroelectronics, STM32Cube HAL and LL driver documentation and example projects for the selected MCU family.

5. ARM, Cortex-M processor technical reference documentation relevant to the core used by the chosen STM32 device.

6. Hitachi, HD44780U LCD controller datasheet, or the datasheet for the actual character LCD controller/module.

7. Servo manufacturer datasheet for operating voltage, pulse range, current demand, and mechanical travel.

8. Course lecture notes, laboratory manual, and instructor-provided circuit diagrams (add the exact title, author, institution, and year).

Appendix A — submission checklist

■ Exact STM32 board and MCU part number recorded. ■ Pin assignments and circuit diagrams included. ■ Code compiled and tested on hardware. ■ Comments and function names are clear. ■ Screenshots or photographs added for all seven tasks. ■ Measured values distinguished from calculated values. ■ Errors and resolutions documented. ■ References checked and formatted according to course requirements.

Appendix B — important implementation note

The code examples in this report are educational templates. The student must adapt GPIO names, timer instances, clock assumptions, LCD driver calls, and UART handles to the actual project generated in STM32CubeIDE. Confirm all electrical limits in the relevant datasheets before connecting hardware.

End of report